



Arquitecturas de aplicaciones web

Universidad de Sevilla

Departamento de Lenguajes y Sistemas
Informáticos

Carlos Arévalo, Daniel Ayala, José Calderón,
Margarita Cruz, Inma Hernández, David Ruiz

UNIVERSIDAD DE SEVILLA

Objetivos



Contenido

Introducción a las arquitecturas

Elementos de una arquitectura software

Estilos de arquitectura web

El framework Silence

Ejemplo Silence



Contenido

Introducción a las arquitecturas

Elementos de una arquitectura software

Estilos de arquitectura web

El framework Silence

Ejemplo Silence

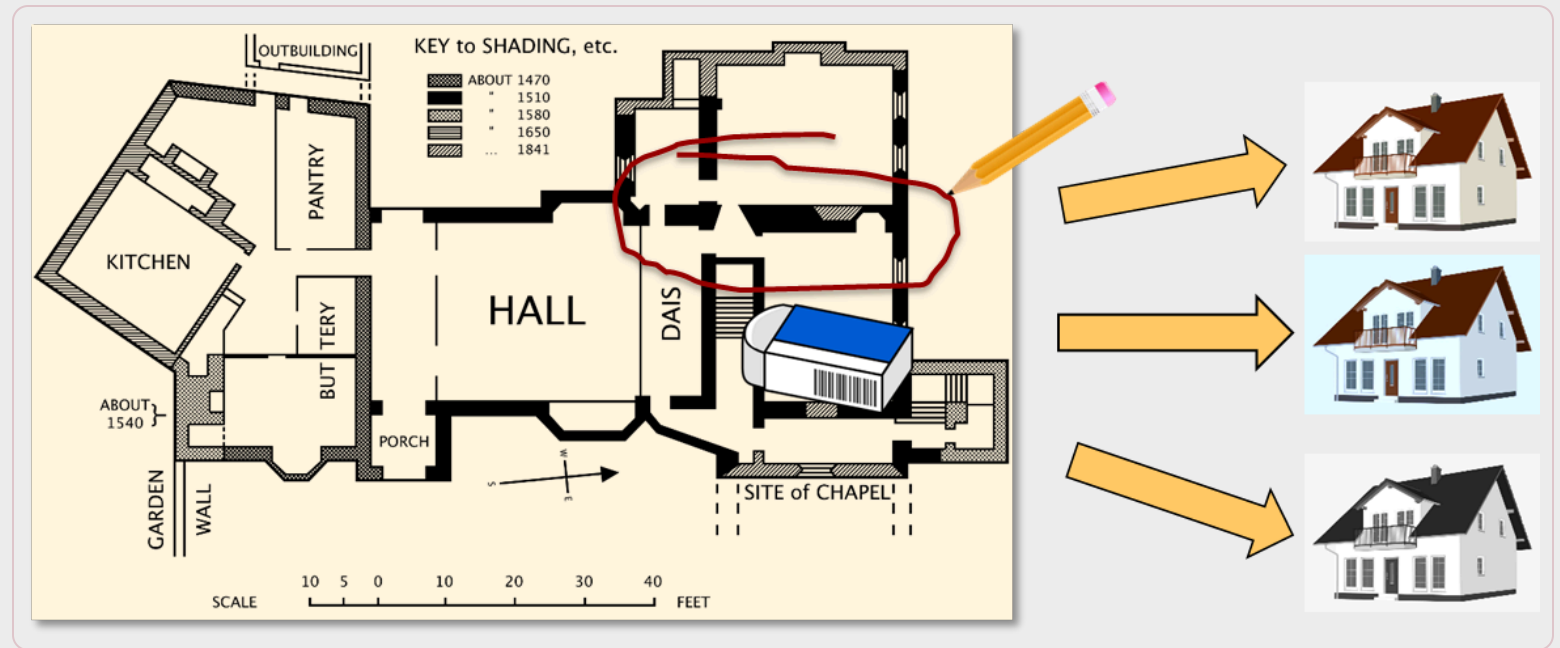


Importancia de las arquitecturas

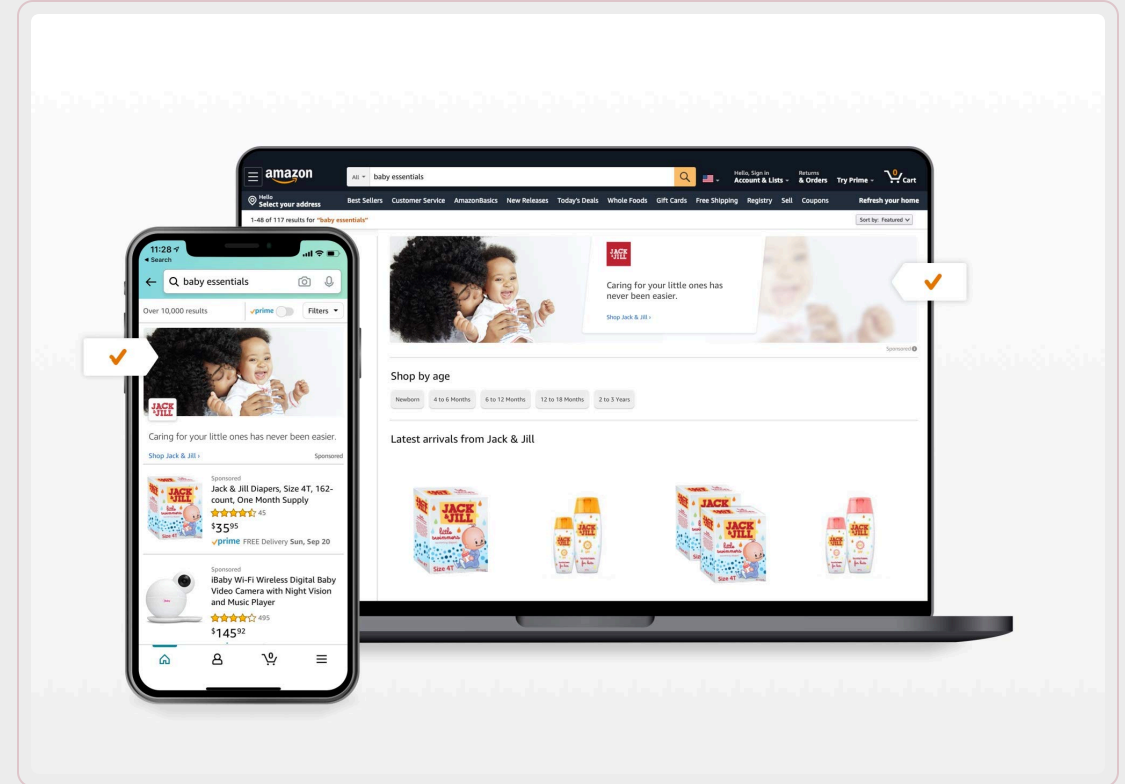
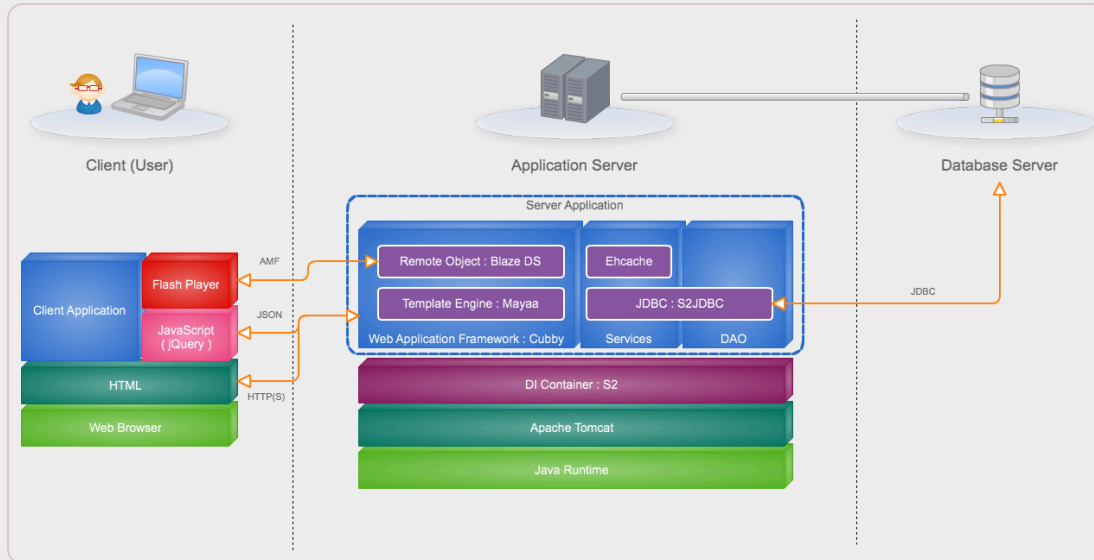
Detección /
corrección temprana

Análisis de la
idoneidad y
completitud

Reutilización de
componentes



Arquitecturas software



Definición de arquitectura

Software architecture refers to the fundamental structures of a software system and the discipline of creating such structures and systems. Each structure comprises software elements, relations among them, and properties of both elements and relations

Fuente: [Wikipedia](#)



Contenido

Introducción a las arquitecturas

Elementos de una arquitectura software

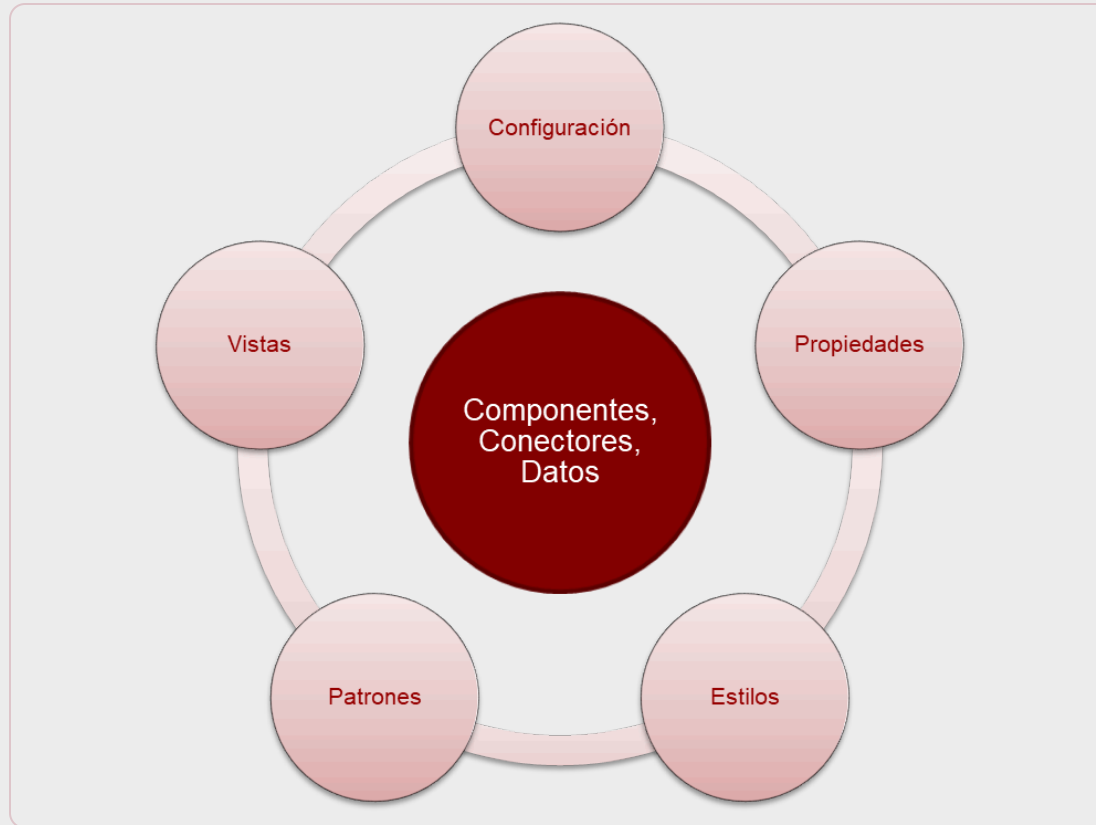
Estilos de arquitectura web

El framework Silence

Ejemplo Silence



Arquitectura software - elementos

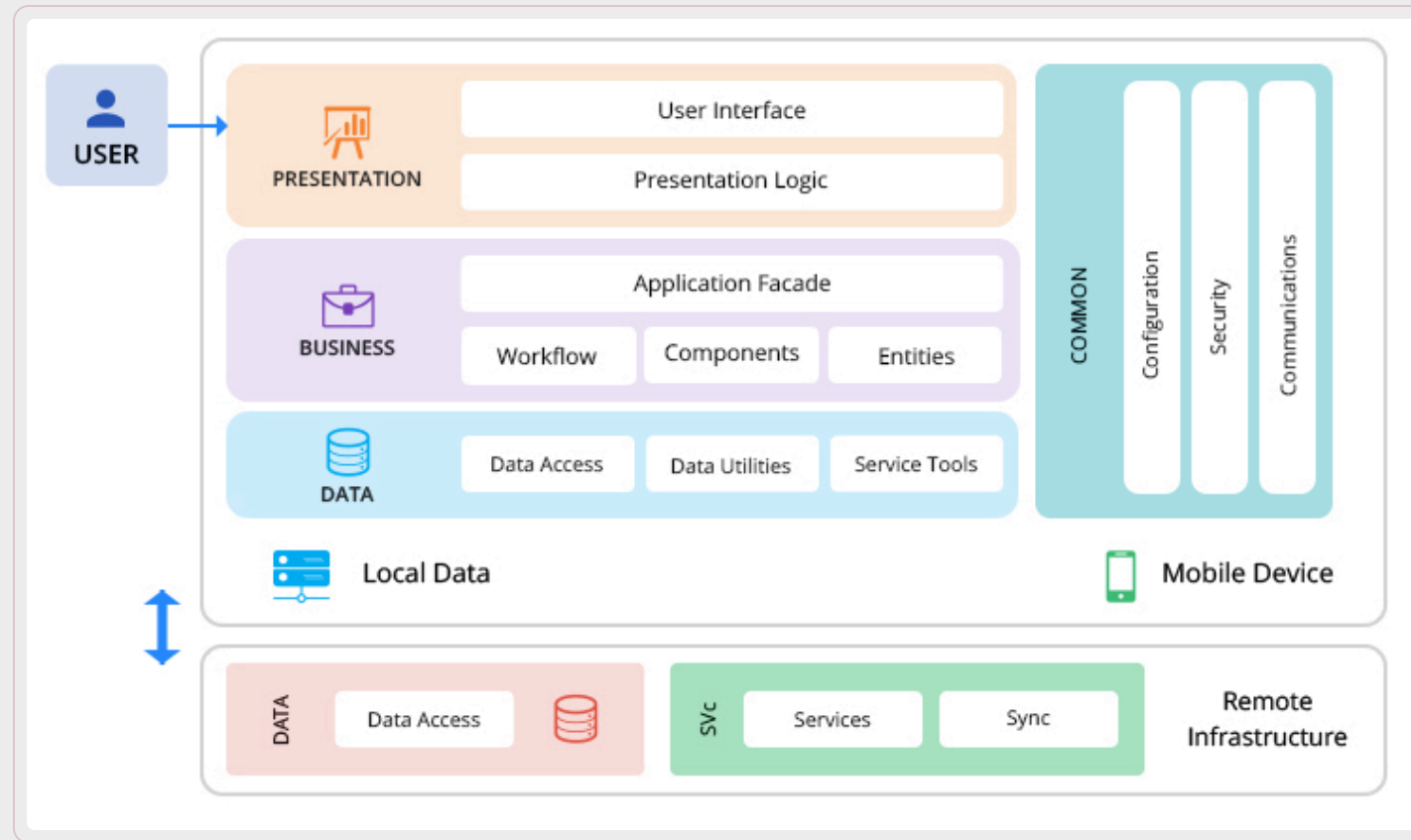


Componente: Unidad abstracta de instrucciones software que transforma datos y ofrece una interfaz

Conector: Mecanismo abstracto que representa comunicación, coordinación o cooperación entre componentes

Datos: Valores que se usan en los componentes y circulan de un componente a otro a través de los conectores

Configuración

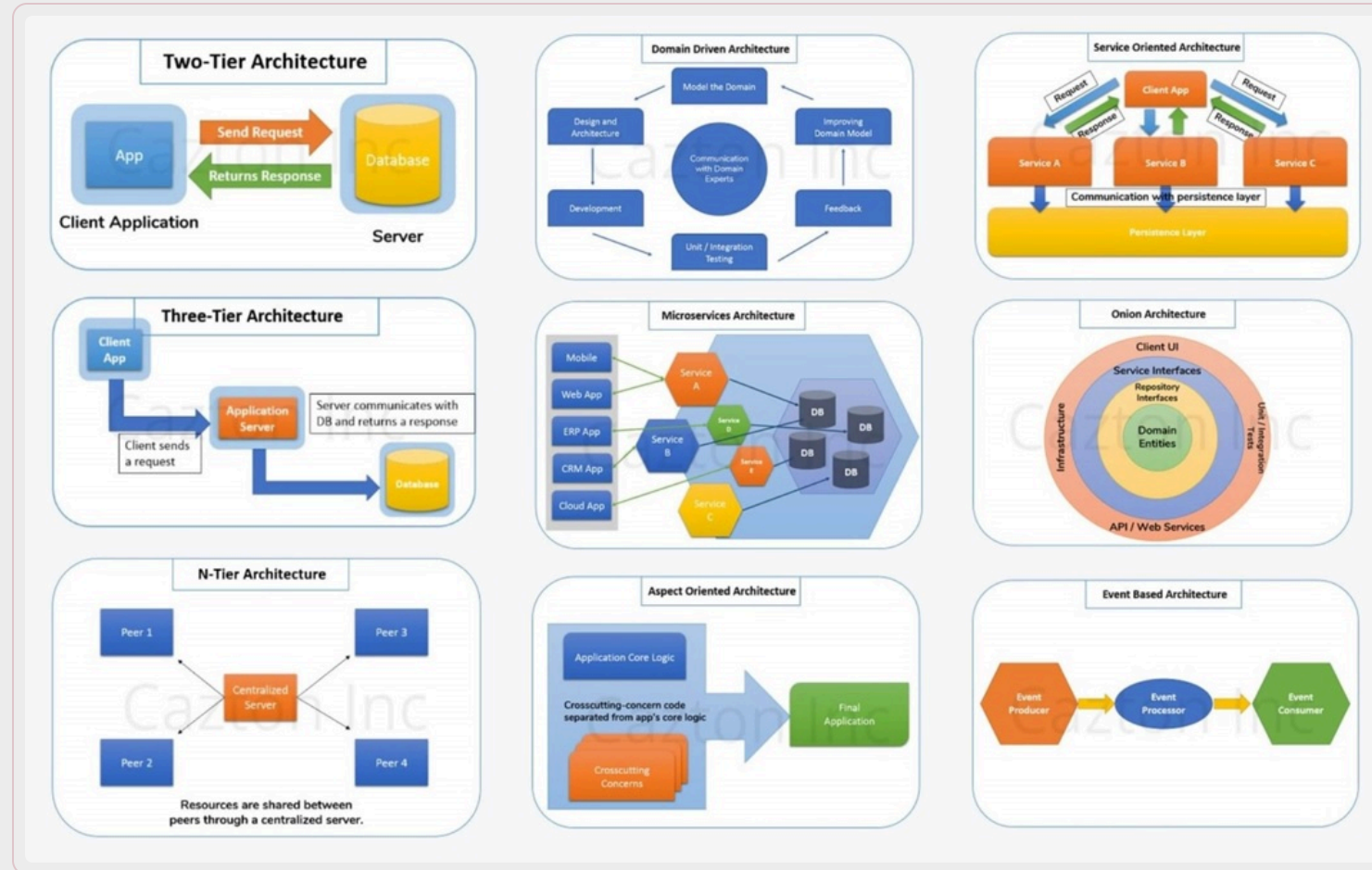


Cómo se disponen e interrelacionan los elementos de la arquitectura

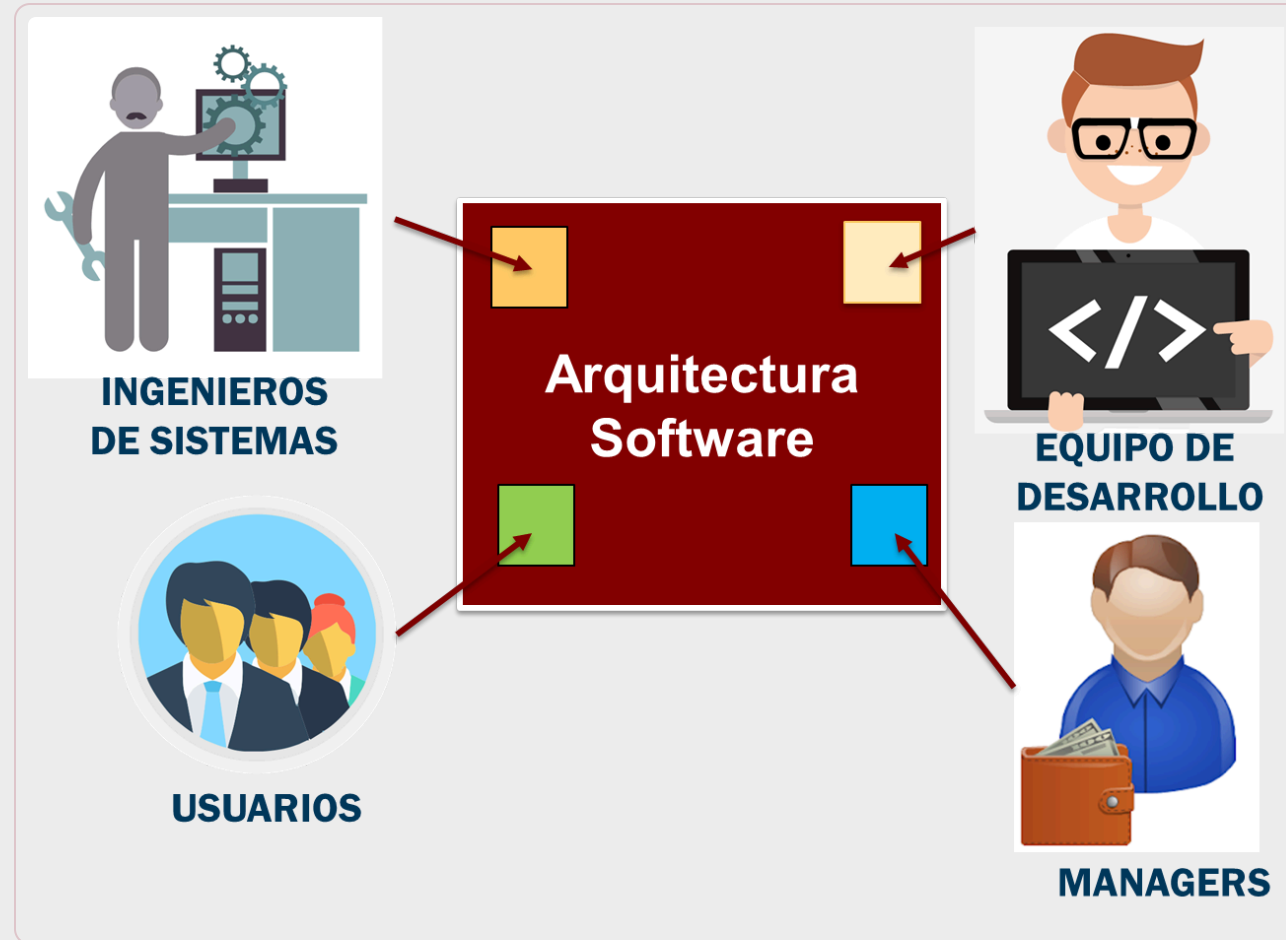
Propiedades



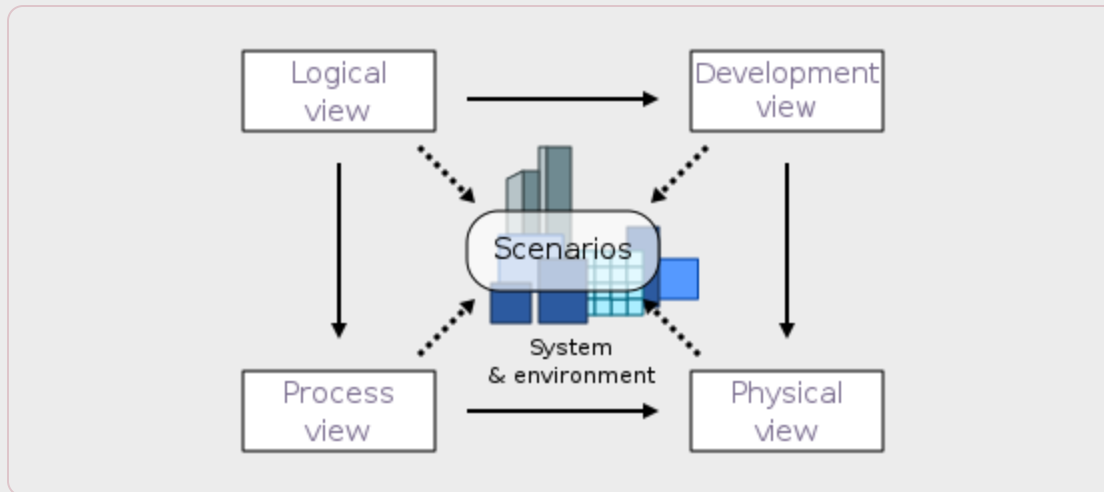
Arquitecturas / Patrones



Vistas



Ejemplo: 4+1



Vista Escenarios

- Casos de uso

Vista lógica

- Funcionalidad para los usuarios finales
- Diagramas de clases y estados

Vista de procesos

- Concurrencia, distribución, performance, escalabilidad,...
- Diagramas de secuencia y actividades

Vista de desarrollo

- Cuestiones de implementación para desarrolladores
- Diagramas de clases y paquetes

Vista física

- Topología de componentes, Comunicaciones, etc. para ingenieros de Sistemas
- Diagramas de despliegue

Contenido

Introducción a las arquitecturas

Elementos de una arquitectura software

Estilos de arquitectura web

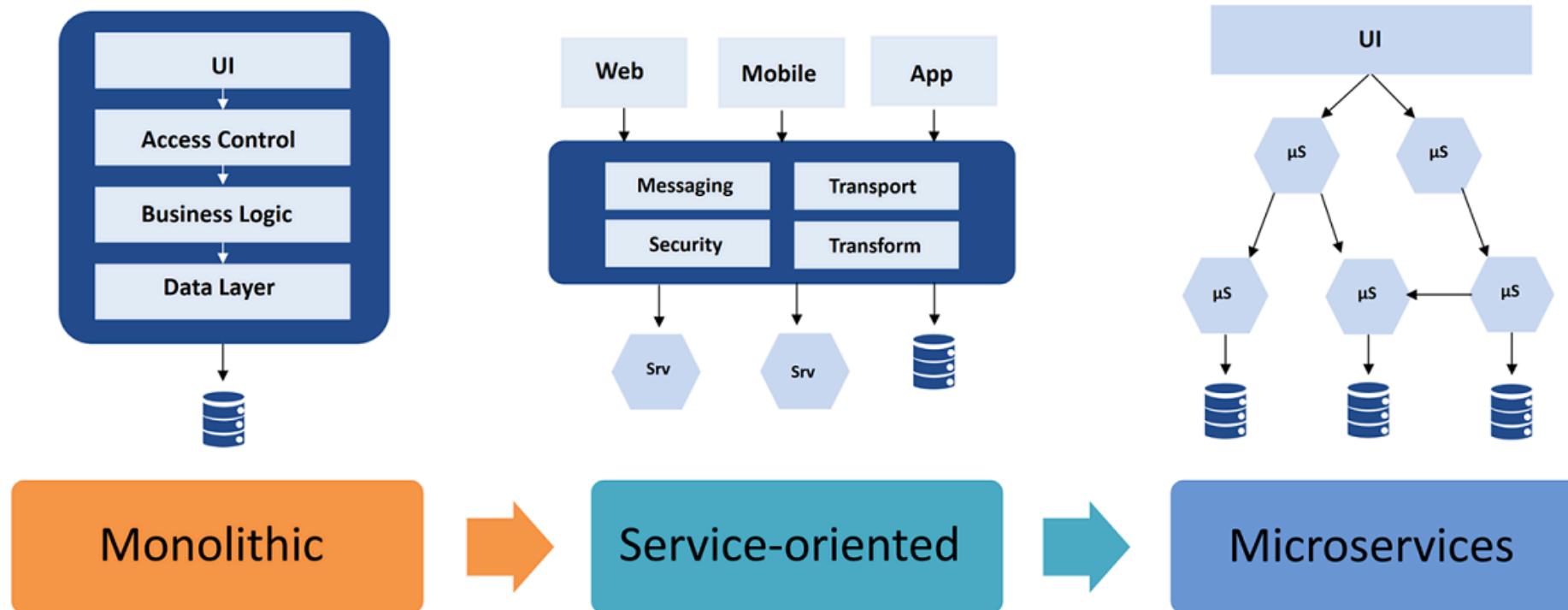
El framework Silence

Ejemplo Silence

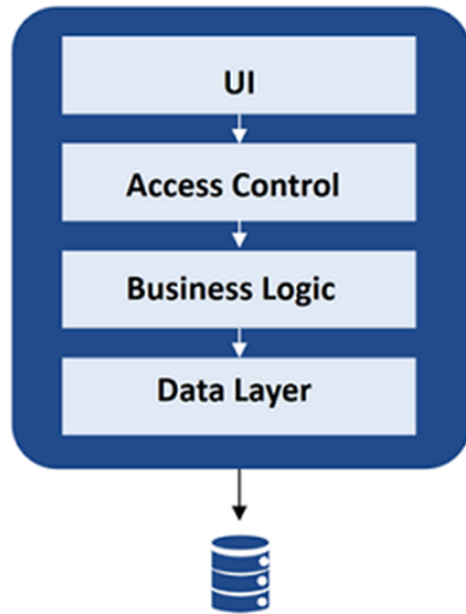


Evolución de las arquitecturas software

Evolution of Software Architectures



Arquitectura monolítica

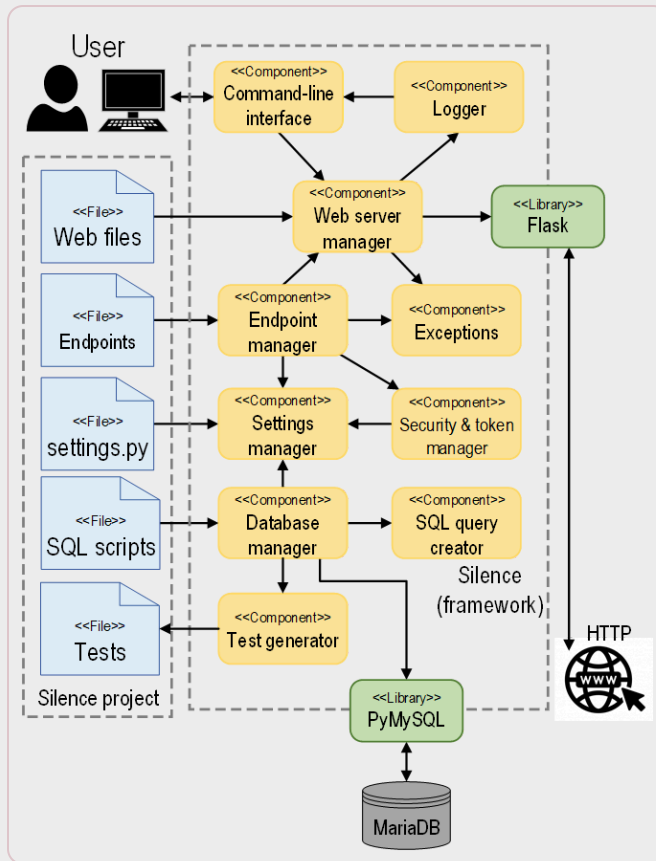


Monolithic

Toda la aplicación en un único programa de una capa, que mezcla interfaz de usuario y acceso a datos.

- **Desventajas:** Sistemas complejos, baja mantenibilidad y escalabilidad
- **Ventajas:** Autonomía e independencia con otros sistemas (autocontenidos)

Arquitecturas basadas en Componentes



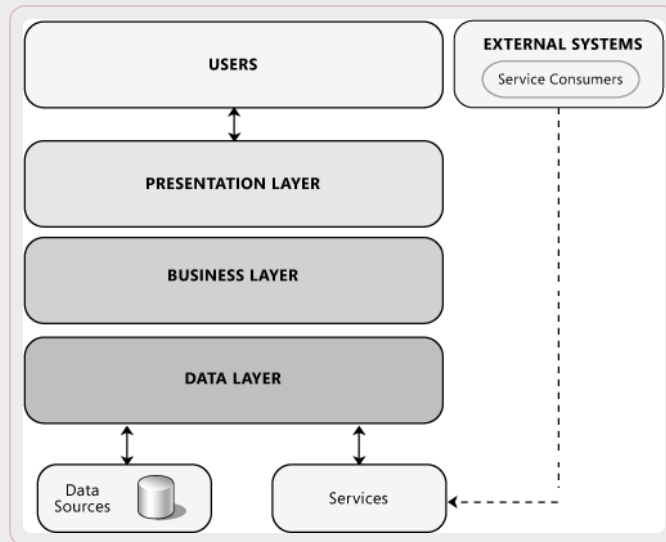
Aplicación estructurada en componentes
(funcionalidades relacionadas)

Componentes intercomunicados con interfaces

Componentes sean reutilizables

- **Ventajas:** cada componente es más fácil de mantener y probar
- **Desventajas:** esfuerzo de diseño previo para definir y separar componentes
- **Ejemplos:** Angular, React, Vue, Silence

Arquitecturas en capas (Caso particular de arquitectura basada en componentes)



Aplicación estructurada en capas

- Cada capa solo se comunica con la capa adyacente
- Cambios afectan a una capa: especialización por capas
- Bajo acoplamiento

Ventajas:

- Escalabilidad
- Tolerancia a fallos

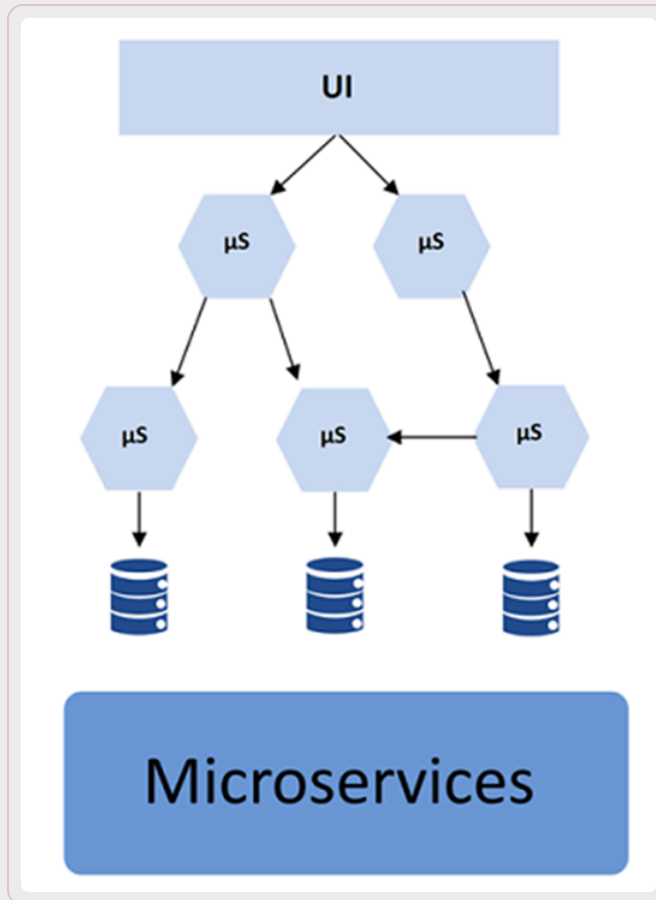
Inconvenientes

- Código redundante en capas distintas
- Modificaciones que producen efectos en cascada
- Rigidez al elegir capas
- Pérdida de eficiencia en el desarrollo en equipo

Ejemplos de capas: Presentación, Lógica de negocio, Acceso a datos, Test

Uso frecuente actual: Django (Python), Spring (Java), Rails (Ruby), Express (JS), Laravel (PHP), ...

Arquitecturas basadas en Microservicios (Caso particular de arquitectura basada en componentes)



Do one thing, and do it well

Aplicación como colección de servicios (componentes) “pequeños” ligeramente acoplados

Cada servicio es implementado usando diferentes lenguajes/plataformas/entornos y desplegado por separado

- **Ventajas:** Modularidad y Escalabilidad mayor que en el caso de Capas. Facilitan el desarrollo distribuido.
- **Desventajas:** Mayor Coste en la comunicación entre procesos, despliegue y testing más complejo, definición de “microservicio” poco clara. Demasiado complejo para aplicaciones pequeñas.
- **Ejemplos:** X, Netflix, Amazon

Contenido

Introducción a las arquitecturas

Elementos de una arquitectura software

Estilos de arquitectura web

El framework Silence

Ejemplo Silence



Silence

Desarrollado en la US con fines docentes

Esquema de trabajo basado en proyectos

Lenguaje Python

Primera versión en 2019: Arquitectura en capas

Segunda versión en 2020: Arquitectura de componentes

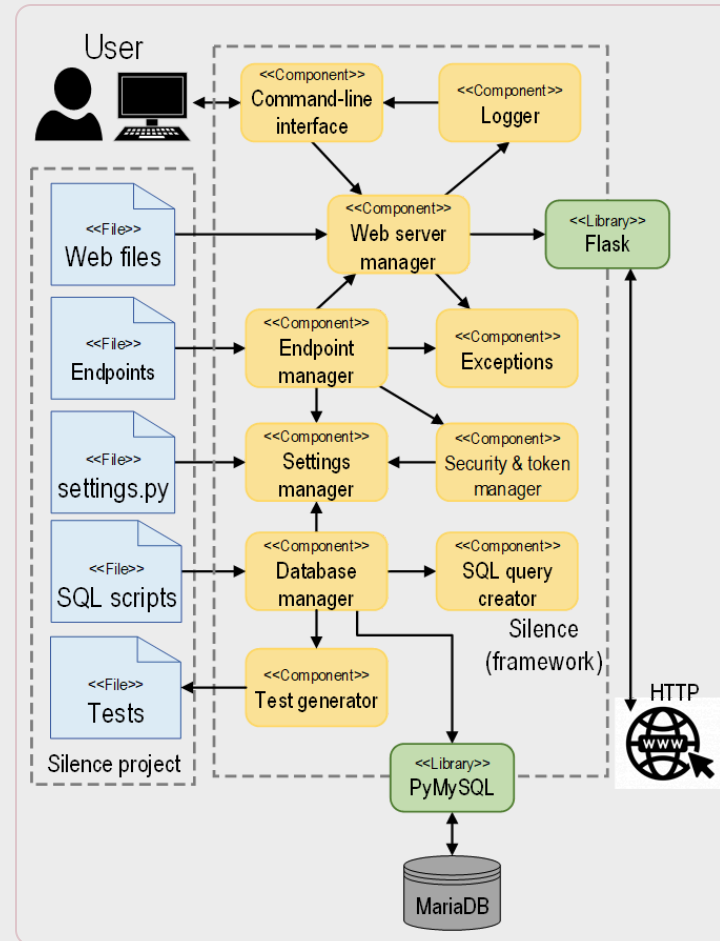
Siguientes versiones con mejoras generales

SILENCE

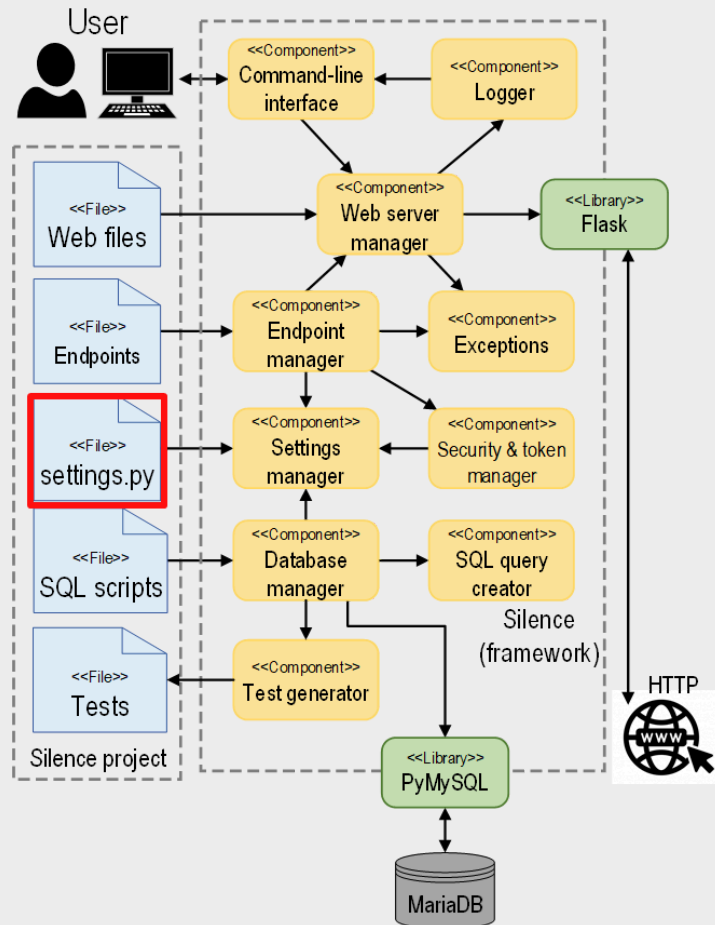


[DEAL-US/Silence](#)

Arquitectura de Silence



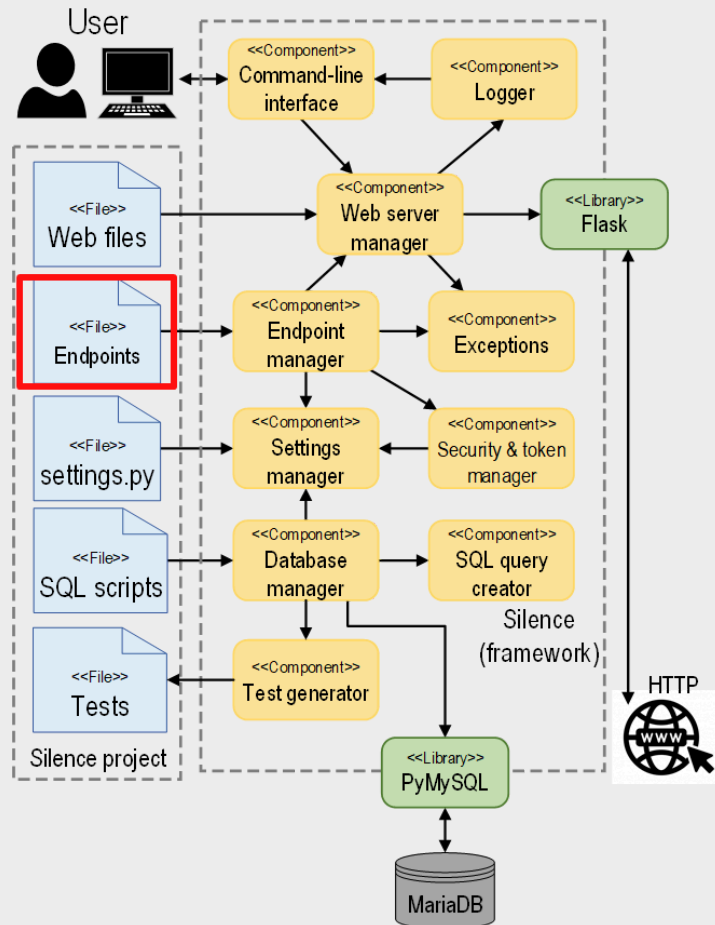
Estructura de un proyecto Silence. settings.py



Configuración del proyecto:
Conexión a la BD, Puerto web, Orden de scripts SQL, Datos para login y registro, ...

```
DB_CONN = {
    "host": "localhost",
    "port": 3306,
    "username": "iissi_user",
    "password": "iissi$user",
    "database": "departments"
}
HTTP_PORT = 8080
SQL_SCRIPTS = [
    "create_tables.sql",
    "populate_database.sql",
]
```

Estructura de un proyecto Silence. endpoints



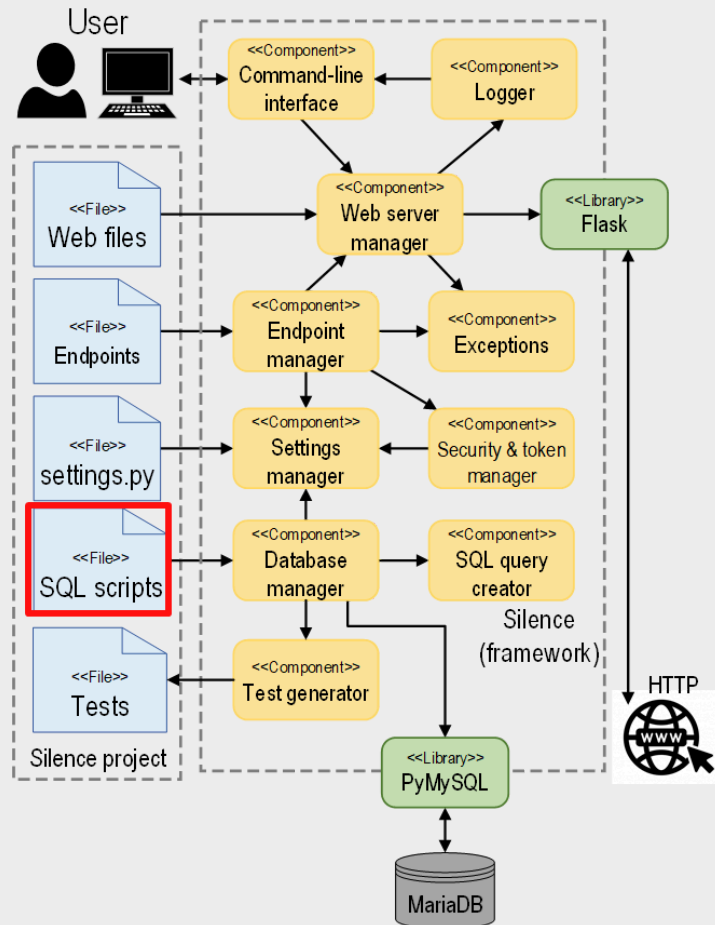
Responsabilidad:

- Establecer correspondencias entre endpoints REST y consultas SQL
- Establecer requisitos de acceso a endpoints

Definidos en formato JSON:

```
"getAll": {
  "route": "/departments",
  "method": "GET",
  "sql": "SELECT * FROM departments",
  "auth_required": false,
  "allowed_roles": ["*"],
  "description": "Gets all departments"
}
```

Estructura de un proyecto Silence. scripts SQL

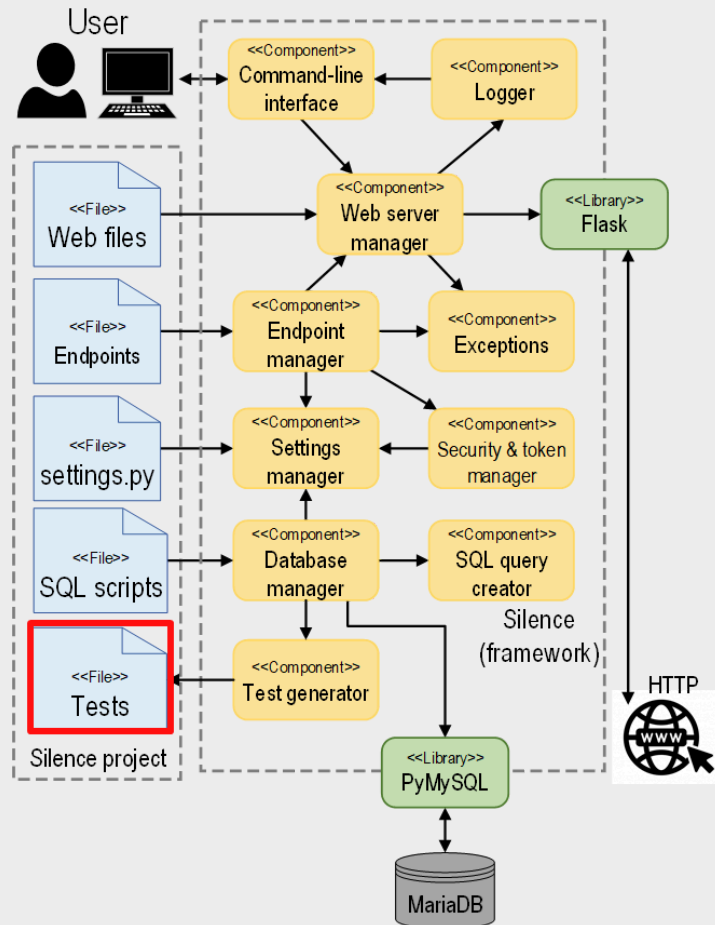


Responsabilidad:

- Crear la estructura de tablas de la BD
- Rellenar las tablas con datos iniciales

```
-- create_tables.sql
CREATE TABLE Departments (
  departmentId INT NOT NULL AUTO_INCREMENT,
  depName VARCHAR(128) NOT NULL,
  city VARCHAR(128)
);
(...)
-- populate_database.sql
INSERT INTO Departments VALUES (...);
```

Estructura de un proyecto Silence. Tests

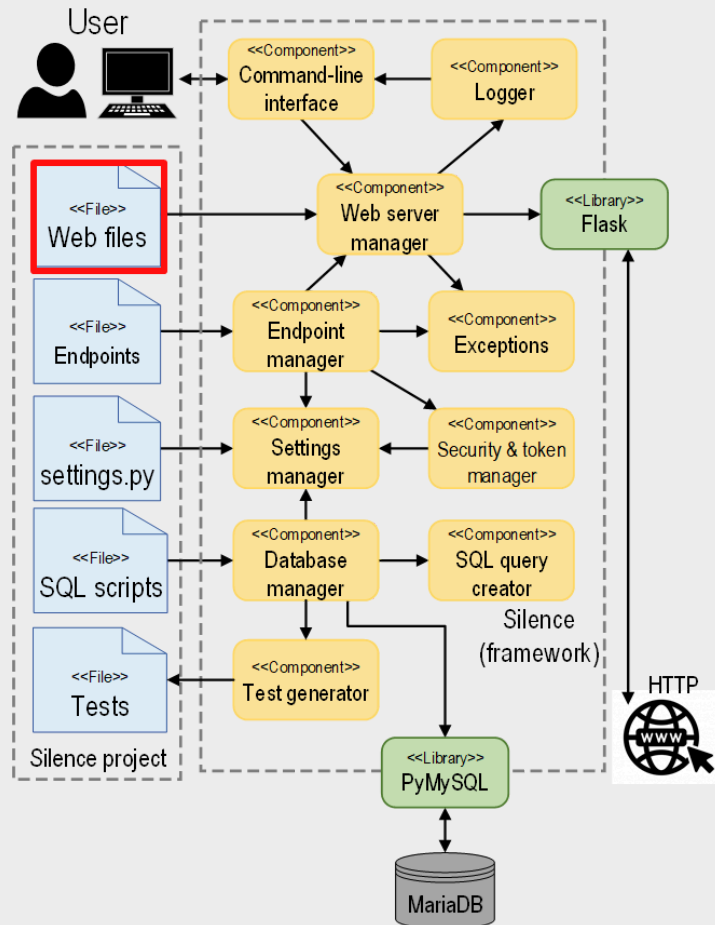


Responsabilidad:

- Comprobar las operaciones contra la BD
- Encontrar bugs en el proyecto

```
POST {{BASE}}/register
Content-Type: application/json
{
  "email": "test_employee_in_dpmt@company.com",
  "password": "123456",
  "firstName": "New",
  "lastName": "Employee",
  "salary": 2000,
  "departmentId": {{deptIdDelete}}
}
```

Estructura de un proyecto Silence. Web

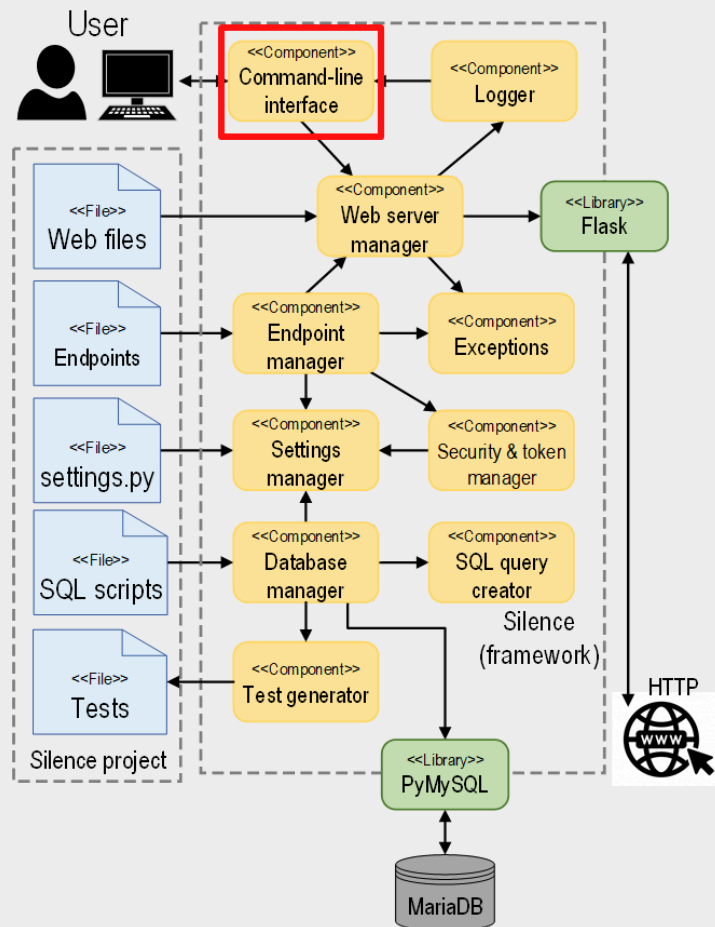


Responsabilidad:

- Crean la capa de presentación e interacción con el usuario
- Archivos HTML, CSS, JavaScript, imágenes...

```
<body>
  <div id="page-header"></div>
  <div class="container">
    <div class="row" id="errors"></div>
    <h1>Employees:</h1>
    <div id="employees"></div>
    ...
  </div>
</body>
```

Estructura del framework Silence. cli

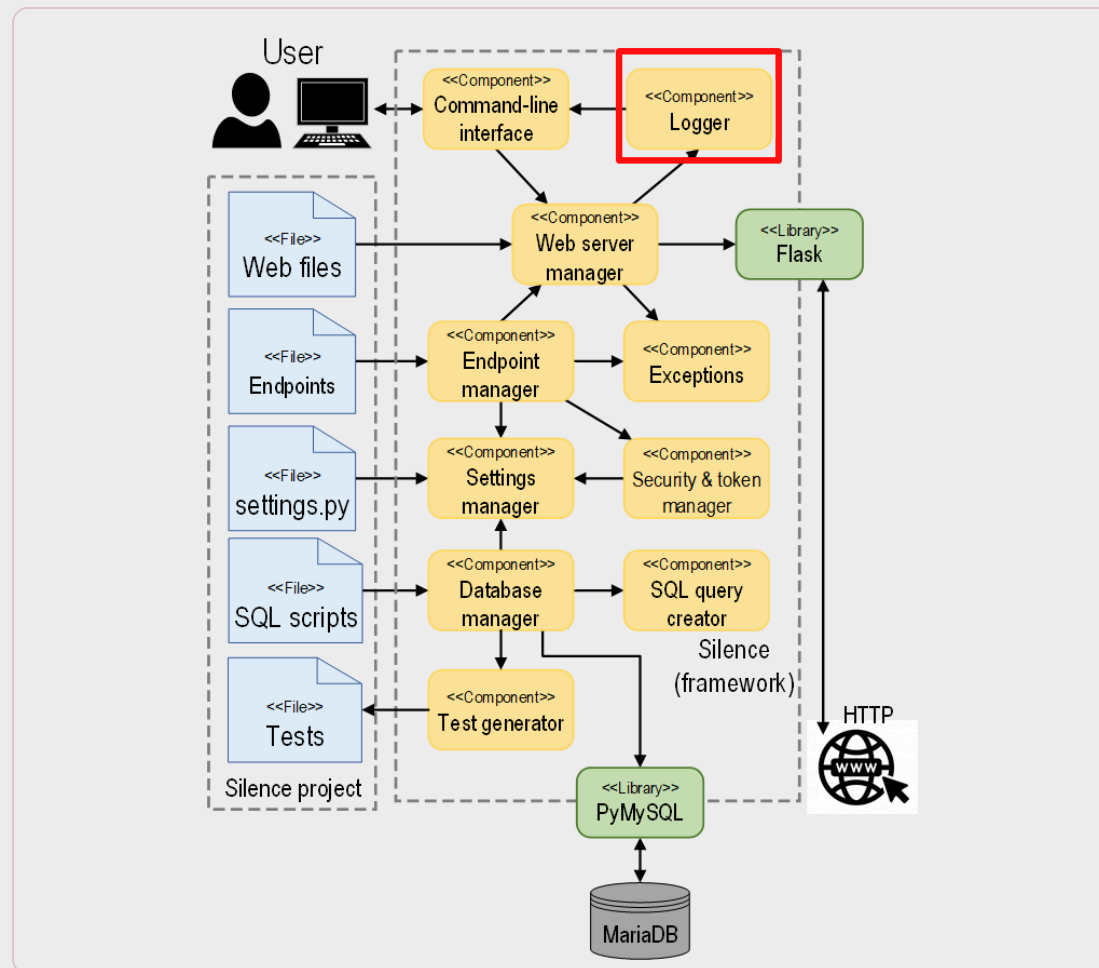


Responsabilidad:

- Provee los comandos de consola para interactuar con Silence

```
C:\Users\usuario\Desktop\silence-app>silence run
Silence v1.0.6
Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
```

Estructura del framework Silence. **Logger**

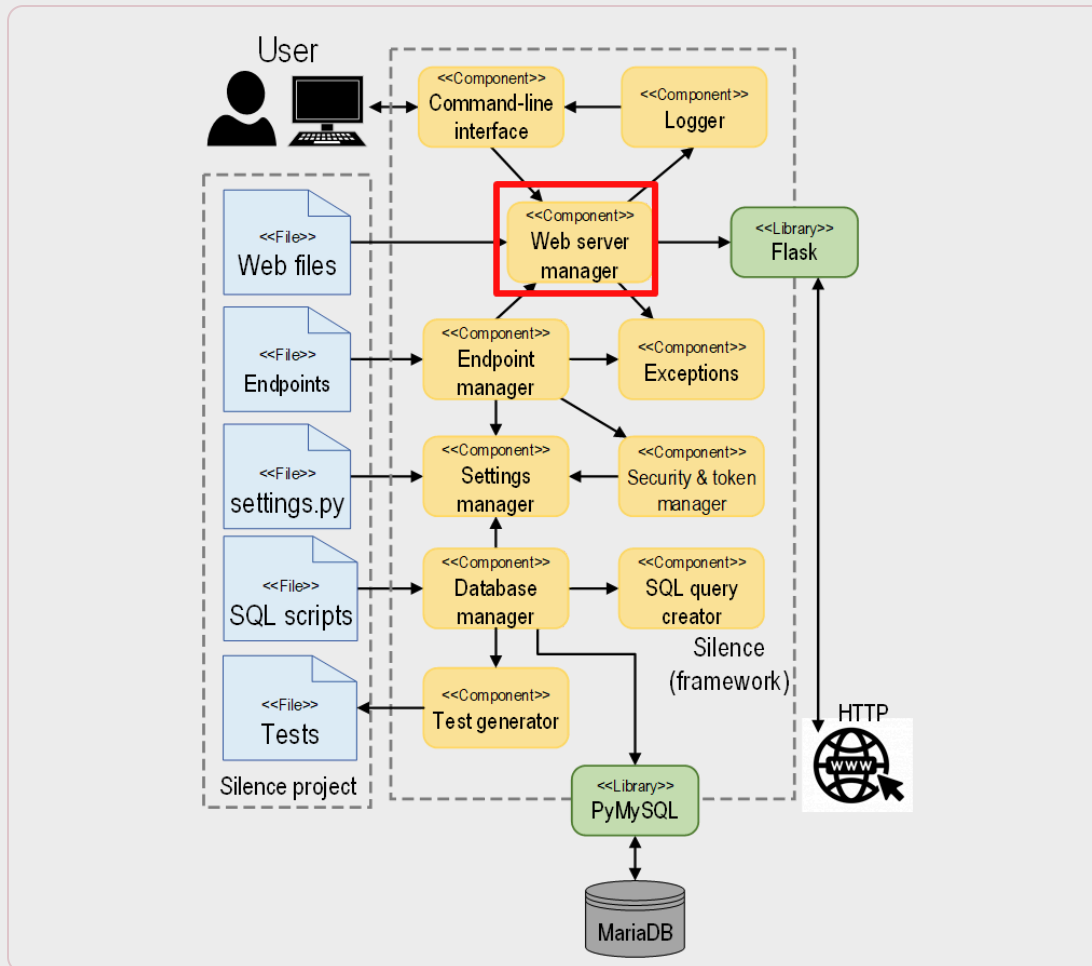


Responsabilidad:

- Mostrar mensajes para el nivel de log configurado
- Unificar el estilo de los mensajes de log

```
20/Nov/2020 11:28:08 | [WEB] GET / from 127.0.0.1 - 304
20/Nov/2020 11:28:08 | [WEB] GET /images/favicon.ico from 127.0.0.1 - 200
20/Nov/2020 11:28:10 | [WEB] GET / from 127.0.0.1 - 200
20/Nov/2020 11:28:10 | [WEB] GET /css/bootstrap.min.css from 127.0.0.1 - 200
20/Nov/2020 11:28:10 | [WEB] GET /css/style.css from 127.0.0.1 - 200
20/Nov/2020 11:28:10 | [WEB] GET /css/font-awesome.min.css from 127.0.0.1 - 200
20/Nov/2020 11:28:10 | [WEB] GET /js/libs/popper.min.js from 127.0.0.1 - 200
20/Nov/2020 11:28:10 | [WEB] GET /js/libs/bootstrap.min.js from 127.0.0.1 - 200
20/Nov/2020 11:28:10 | [WEB] GET /js/utls/include.js from 127.0.0.1 - 200
20/Nov/2020 11:28:10 | [WEB] GET /js/libs/jquery-3.4.1.min.js from 127.0.0.1 - 200
20/Nov/2020 11:28:10 | [WEB] GET /js/header.js from 127.0.0.1 - 200
20/Nov/2020 11:28:10 | [WEB] GET /js/libs/axios.min.js from 127.0.0.1 - 200
20/Nov/2020 11:28:10 | [WEB] GET /js/index.js from 127.0.0.1 - 200
20/Nov/2020 11:28:10 | [WEB] GET /header.html from 127.0.0.1 - 200
```

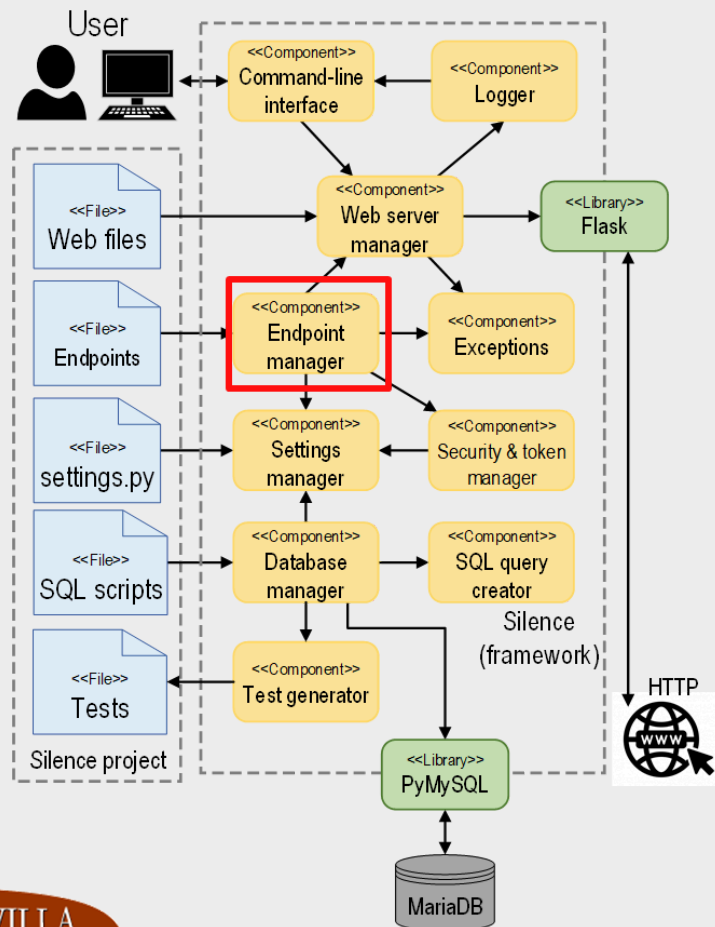
Estructura del framework Silence. web server



Responsabilidad:

- Configurar e iniciar el servidor web
- Registrar los endpoints declarados

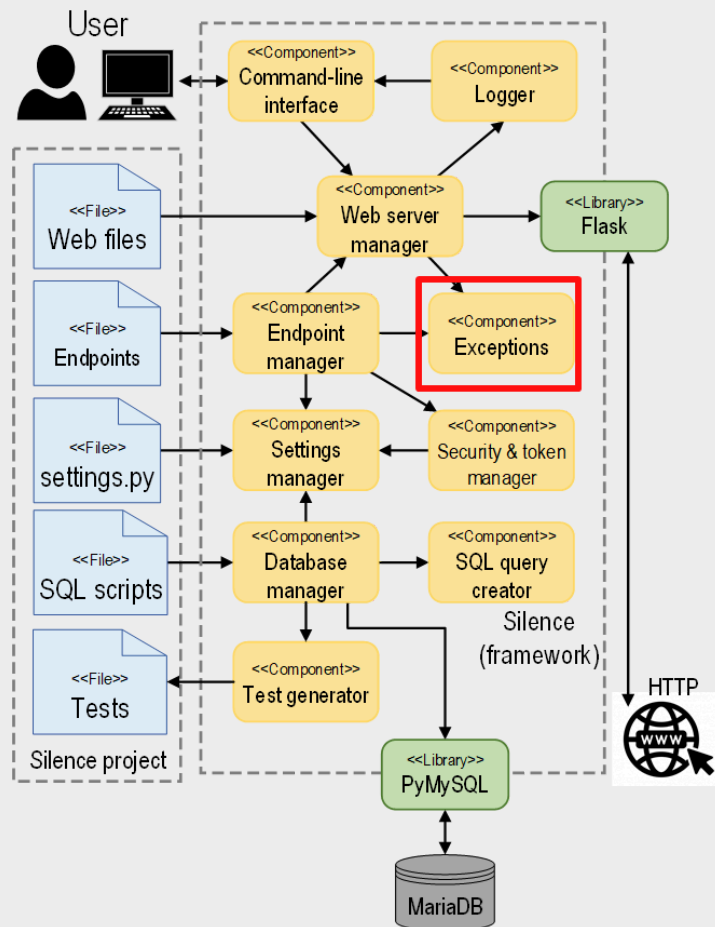
Estructura del framework Silence. **gestor endpoints**



Responsabilidad:

- Leer los endpoints declarados por el usuario y generar los endpoints automáticos
- Registrar los endpoints en el servidor web

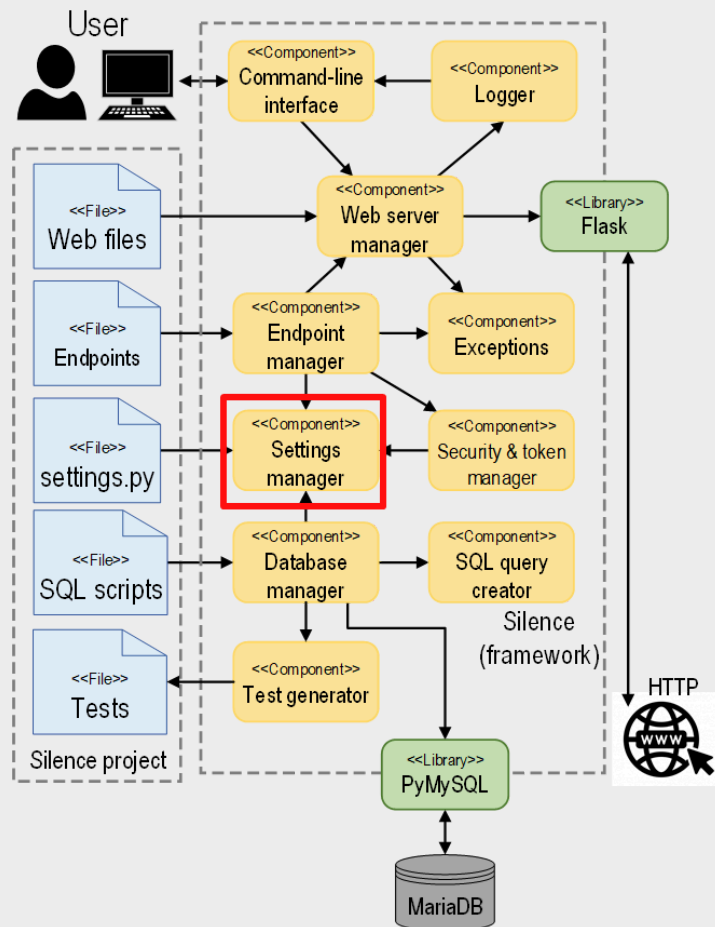
Estructura del framework Silence. excepciones



Responsabilidad:

- Proveer las excepciones y warnings usados por Silence

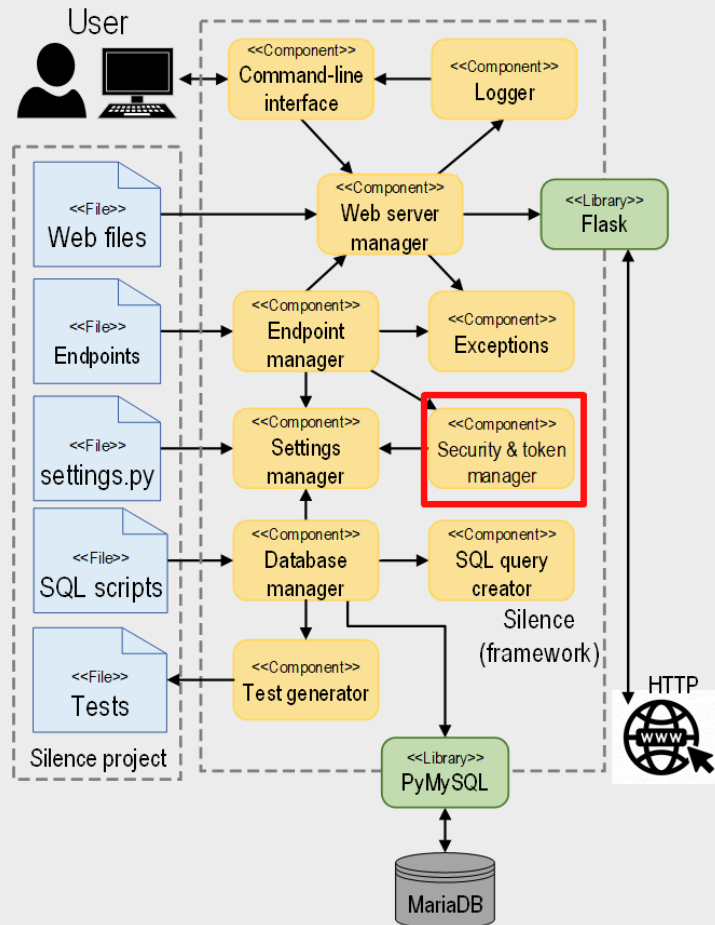
Estructura del framework Silence. **setting man**



Responsabilidad:

- Leer y aplicar los ajustes del proyecto
- Proveer valores por defecto

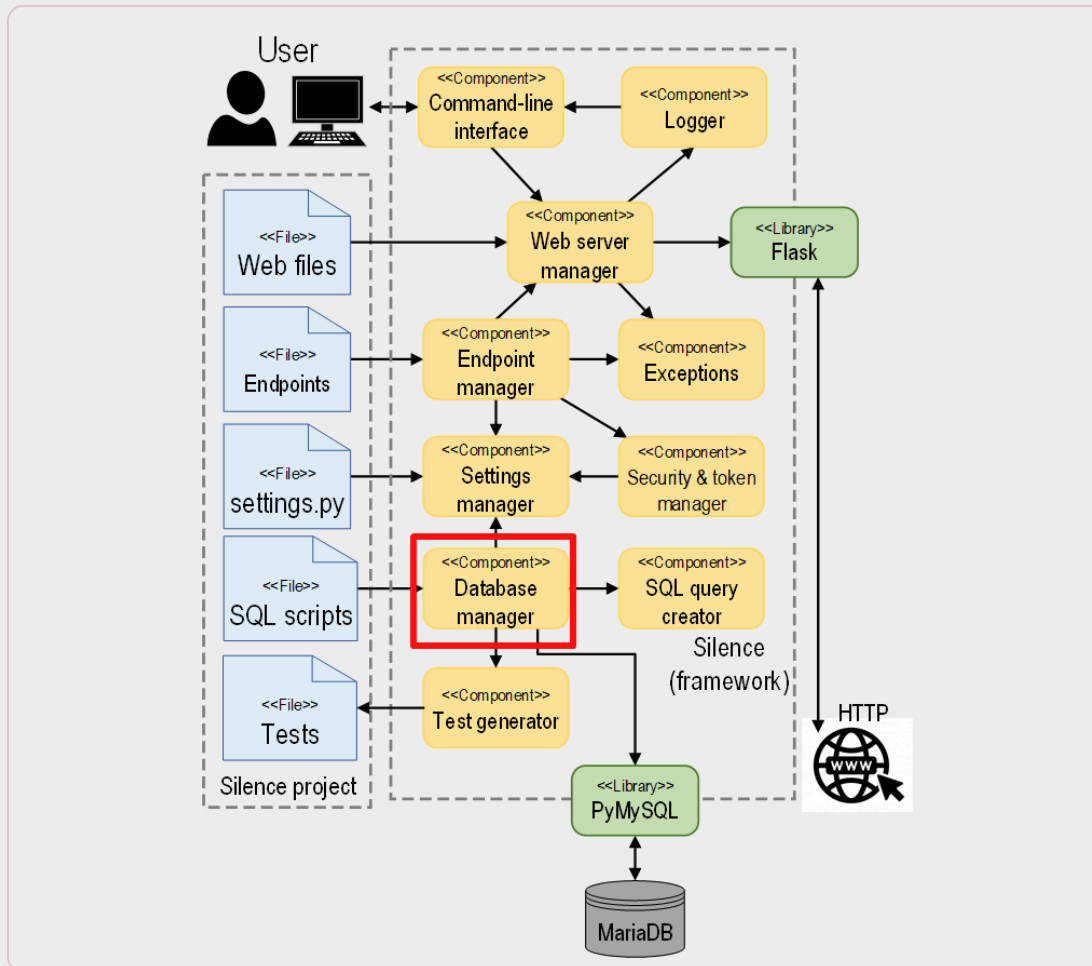
Estructura del framework Silence. security



Responsabilidad:

- Aplicar ajustes de seguridad
- Crear y comprobar tokens de sesión

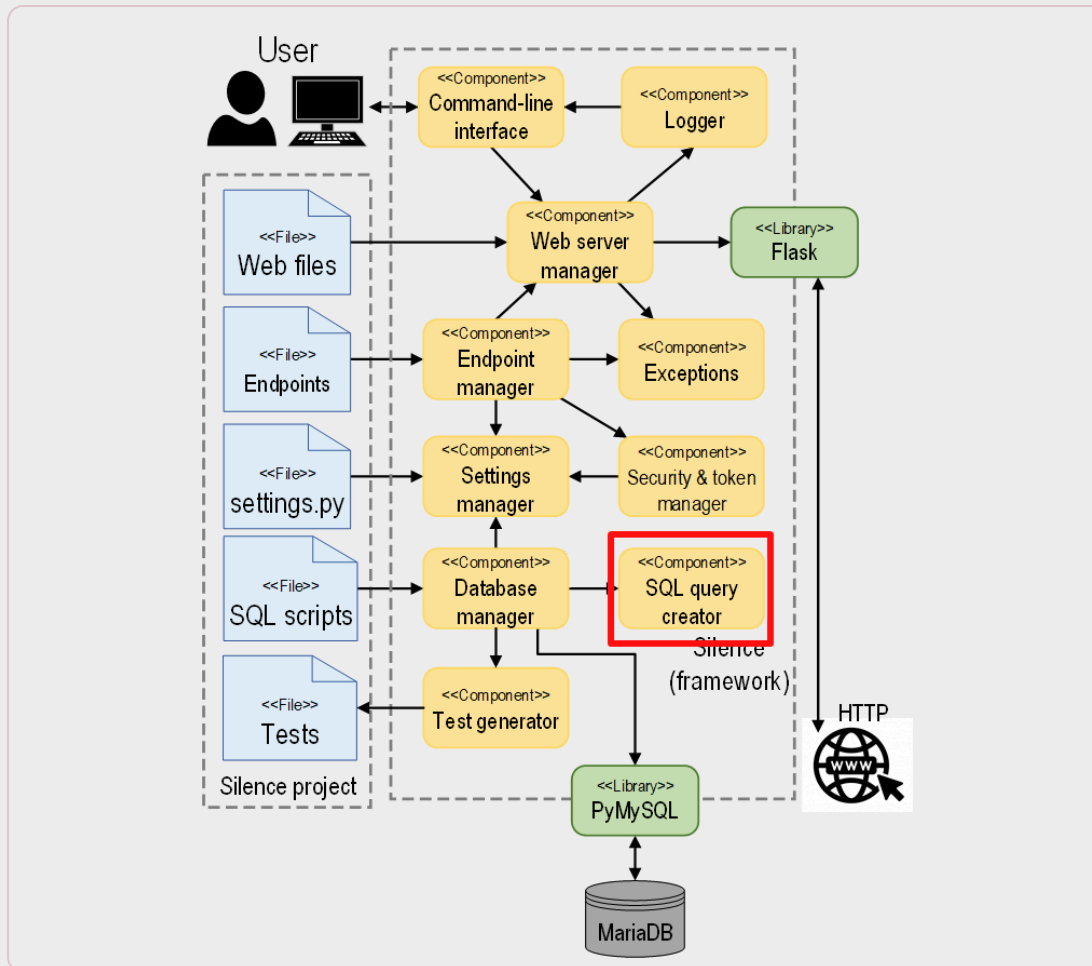
Estructura del framework Silence. db manager



Responsabilidad:

- Conectar con la base de datos
- Ejecutar scripts SQL
- Proveer conexiones a la BD
- Abstraer el SGBD concreto

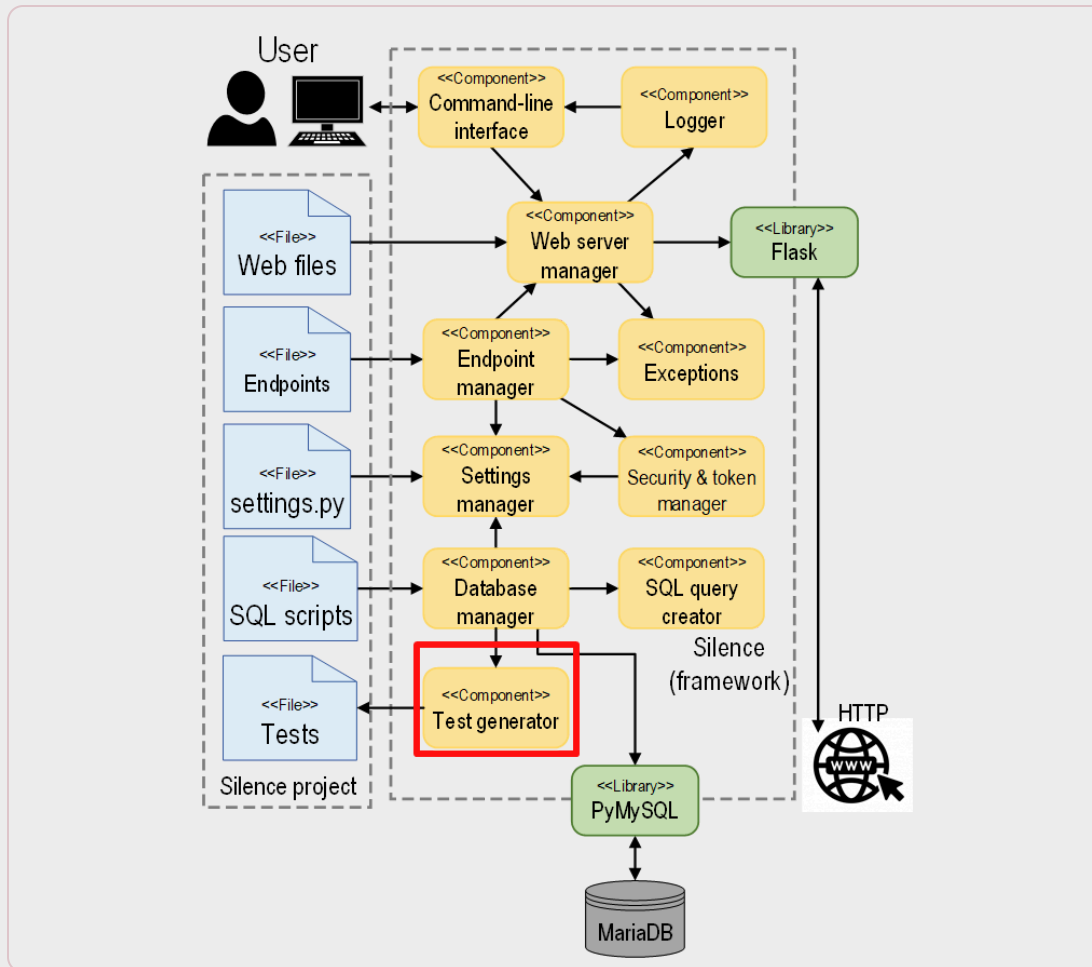
Estructura del framework Silence. query creator



Responsabilidad:

- Crear programáticamente consultas SQL

Estructura del framework Silence. test gen



Responsabilidad:

- Crear automáticamente tests para las tablas en la BD
- Escribir los archivos de test autogenerados

Contenido

Introducción a las arquitecturas

Elementos de una arquitectura software

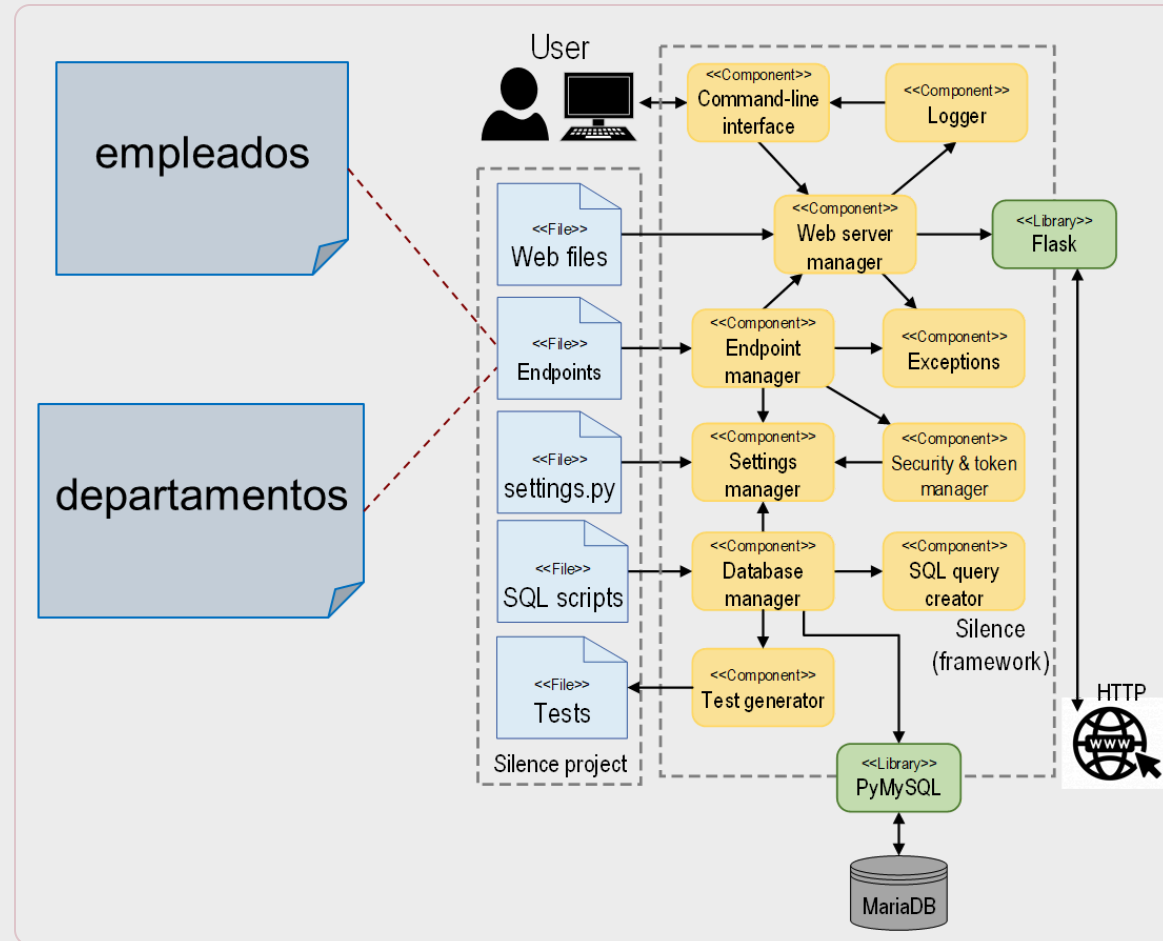
Estilos de arquitectura web

El framework Silence

Ejemplo Silence



Ficheros de endpoints



Endpoints de Departments

```
{
  "getAll": {
    "route": "/departments",
    "method": "GET",
    "sql": "SELECT * FROM departments",
    "auth_required": false,
    "description": "Gets all departments"
  },
  "getById": {
    "route": "/departments/$departmentId",
    "sql": "SELECT * FROM departments
      WHERE departmentId = $departmentId",
    "description": "Gets an entry from 'departments'
      by its primary key"
  },
}
```

```
  "create": {
    "method": "POST",
    "sql": "INSERT INTO departments(name, city)
      VALUES ($name, $city)",
    "auth_required": true,
    "description": "Creates a new entry in
      'departments'",
    "request_body_params": [
      "name",
      "city"
    ],
  },
  "delete": {
    "method": "DELETE",
    "sql": "DELETE FROM departments
      WHERE departmentId = $departmentId",
    "description": "Deletes an existing entry in
      'departments' by its primary key"
  },
}
```

Endpoints de Employees

```
{
  "getAll": {
    "route": "/employees",
    "method": "GET",
    "sql": "SELECT * FROM employees",
    "auth_required": false,
    "description": "Gets all employees"
  },
  "getById": {
    "route": "/employees/$employeeId",
    "sql": "SELECT * FROM employees
      WHERE employeeId = $employeeId",
    "description": "Gets an entry from 'employees'
      by its primary key"
  }
}
```

```
"create": {
  "method": "POST",
  "sql": "INSERT INTO employees
    (email, password, departmentId, bossId,
    firstName, lastName, salary)
  VALUES
    ($email, $password$departmentId, $bossId,
    $firstName, $lastName, $salary)",
  "auth_required": true,
  "allowed_roles": ["*"],
  "description": "Creates a new entry in 'employees'",
  "request_body_params": ["email", "password",
    "departmentId", "bossId", "firstName",
    "lastName", "salary" ]
}
```

Conclusiones

Las **arquitecturas software** permiten razonar sobre la estructura del sistema antes de implementarlo y ayudan a mejorar su **mantenibilidad y evolución**.

Una arquitectura se describe mediante **componentes, conectores, datos y su configuración**, y puede analizarse desde distintas **vistas**.

En aplicaciones web, los estilos **monolítico, por componentes, en capas y de microservicios** responden a compromisos distintos de **desacoplamiento, escalabilidad y complejidad**.

Conclusiones

Silence es un framework docente que materializa una arquitectura basada en **componentes**, separando **configuración**, **endpoints**, **scripts SQL**, **tests** y **capa web**.

El ejemplo final muestra cómo una arquitectura web conecta el **modelo de datos**, la **API REST** y la **implementación** concreta del proyecto para construir una aplicación coherente.





Gracias

Universidad de Sevilla

**Departamento de Lenguajes y Sistemas
Informáticos**

UNIVERSIDAD DE SEVILLA